

Agent Bash 工具的顶层设计方法论

一、首先重新定义 Bash Tool

设计 Bash 工具时，最容易犯的错误是把它理解成：

```
输入一段命令
→ 启动 Shell
→ 返回 stdout
```

这只能算一个 Shell API，不能算生产级 Agent Tool。

在 Agent 系统中，Bash Tool 应被定义为：

一个将模型生成的命令意图，转换成受解析、受授权、受隔离、可观测、可中断、可恢复的系统操作的执行控制平面。

它至少需要解决四类问题：

1. **语义问题**：模型到底想执行什么？
2. **安全问题**：这条命令是否允许执行？
3. **运行问题**：命令应该以前台、后台还是沙箱方式运行？
4. **上下文问题**：哪些执行结果应该返回给模型？

因此 Bash Tool 不是一个独立函数，而应当被设计为一个小型 Runtime。

二、先明确 Bash Tool 的职责边界

1. Bash 不应该替代所有专用工具

即使 Bash 可以完成文件读取、搜索、编辑和 Git 操作，也不应该让 Agent 默认全部使用 Bash。

更合理的工具体系是：

```
结构化、常用、低风险操作
→ 使用专用工具

开放式、组合式、CLI 驱动操作
→ 使用 Bash 工具
```

例如：

读取文件	→ Read
修改文件	→ Edit
搜索文本	→ Grep
匹配路径	→ Glob
执行测试	→ Bash
运行构建	→ Bash
调用第三方 CLI	→ Bash
启动服务	→ Bash

专用工具能够提供更小的权限范围、更清晰的 Schema 和更可控的副作用；Bash 更适合作为通用执行能力和兜底能力。原文也强调应优先使用专用工具，并遵循最小权限原则。

因此第一条方法论是：

先设计工具边界，再设计 Bash。不要因为 Bash 能做所有事，就让它承担所有事。

三、采用“控制面 + 执行面”架构

一个生产级 Bash Tool 可以在逻辑上划分为两个平面。

1. 控制面

控制面不真正执行命令，负责回答：

- 命令是什么？
- 命令包含哪些执行单元？
- 命令是否危险？
- 是否需要用户批准？
- 应使用哪种沙箱策略？
- 应以前台还是后台运行？
- 输出应该如何进入上下文？

控制面包含：

```
Command Parser
Policy Engine
Permission Engine
Approval Manager
Execution Planner
Task Manager
Context Output Manager
```

2. 执行面

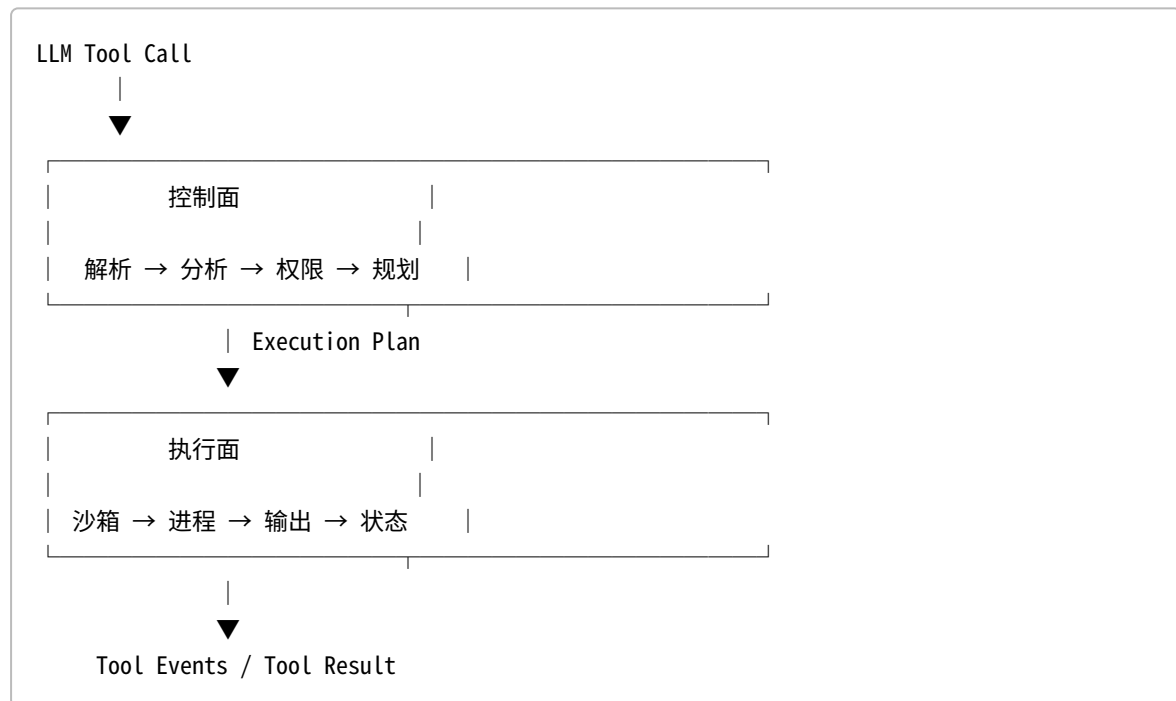
执行面负责真正运行命令，包括：

Process Launcher
Sandbox Runtime
Process Supervisor
Output Collector
Resource Limiter
Artifact Storage

这样分层的好处是：

- 权限判断不会和进程实现耦合；
- 沙箱可以替换；
- 前台和后台执行可以共享同一个执行器；
- 用户批准的是固定执行计划，而不是重新生成的命令；
- 测试时可以独立验证策略层。

整体结构可以表示为：



四、核心对象不应该只是 Command String

如果系统中唯一的一等对象是：

"npm test && git status"

后续权限、审计、后台任务和错误恢复都会很难设计。

更合理的做法是引入几个核心领域对象。

1. Command Request

表示模型提出的原始请求，包括：

- 原始命令；
- 命令用途描述；
- 工作目录；
- 超时时间；
- 是否允许后台运行；
- 是否请求绕过沙箱；
- 当前会话、项目和用户身份。

2. Parsed Command

表示解析后的执行语义，包括：

- 命令及参数；
- 管道和顺序关系；
- 重定向；
- 环境变量；
- Wrapper；
- 动态展开；
- 文件路径；
- 网络、进程和文件系统意图。

3. Execution Plan

表示系统真正准备执行的计划，包括：

- 规范化后的命令；
- 实际工作目录；
- 前台或后台模式；
- 沙箱配置；
- 文件系统权限；
- 网络权限；
- 资源限制；
- 超时策略；
- 审批结果；
- 输出策略。

4. Task

表示一次正在运行或已经运行的任务，包括：

- Task ID；
- Execution Plan；
- 当前状态；
- 进程身份；
- 开始和结束时间；
- 退出码；

- 输出位置；
- 是否被中断；
- 是否因沙箱失败；
- 审计信息。

核心原则是：

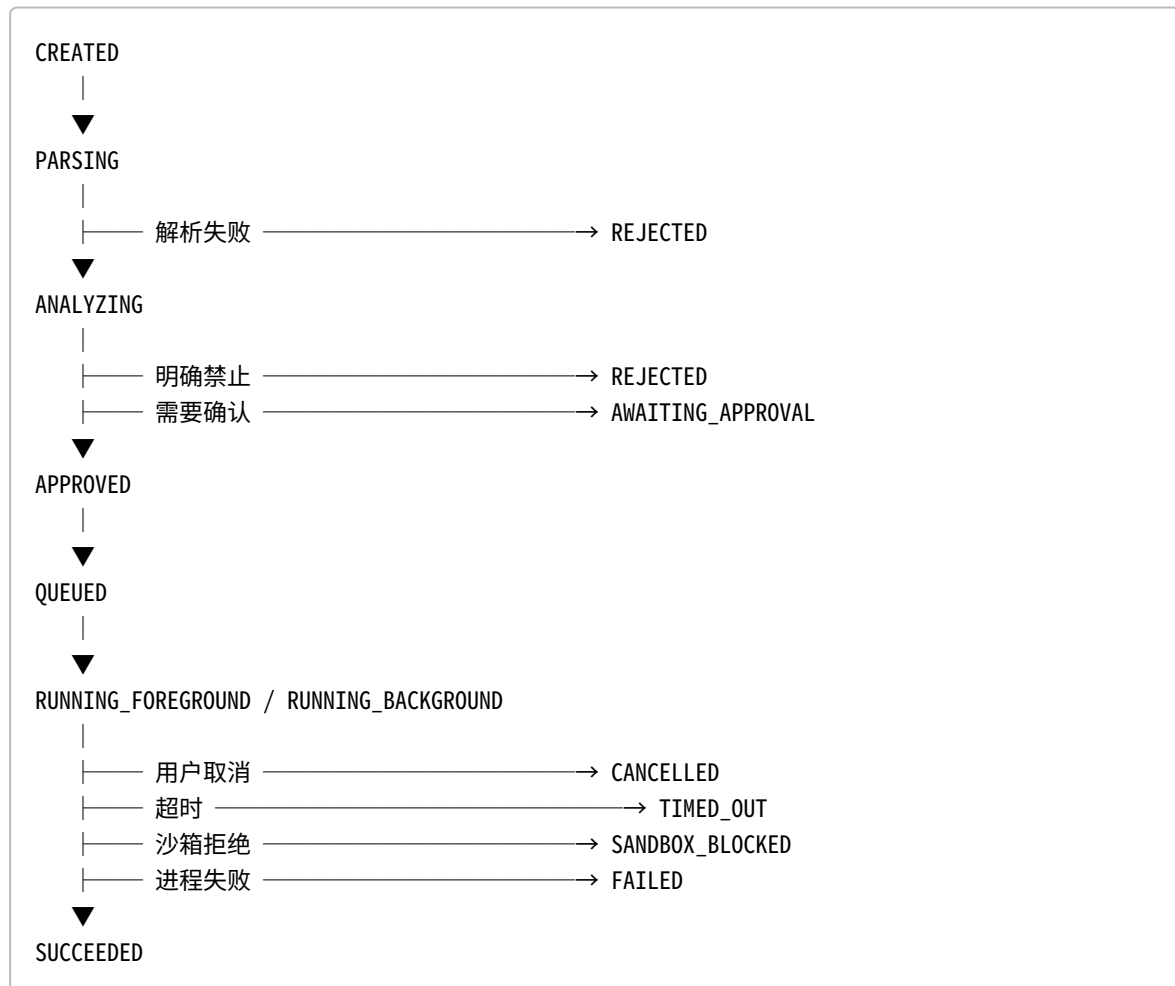
模型提交的是 Command Request，系统批准并执行的是不可变的 Execution Plan。

这能够避免用户批准命令 A 后，模型或系统重新生成命令 B 再执行。

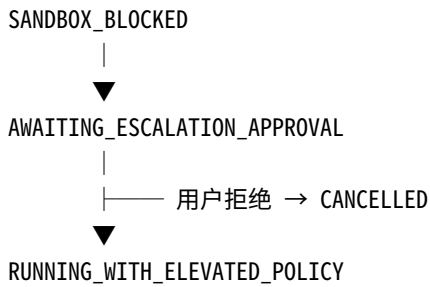
五、把 Bash 执行设计成状态机

Bash Tool 不应只有“执行中”和“执行完成”两个状态。

建议使用完整的任务状态机：



如果沙箱阻止了命令，还可以增加受控升级流程：



状态机的价值在于：

- Agent 能准确理解任务当前阶段；
- 前后台任务共享统一语义；
- 用户取消和超时不再被误认为普通失败；
- 沙箱权限不足可以被单独识别；
- 重试策略可以基于失败类型制定。

六、命令解析：从字符串升级为执行图

1. 不应只做字符串匹配

下面这些命令不能只看开头：

```

安全命令 | 危险命令
timeout 10 危险命令
env KEY=value 危险命令
cd 某目录 && 执行命令
  
```

因此应将 Shell 命令解析为结构化执行图：

```

Sequence
├── Command A
├── Pipe
│   ├── Command B
│   └── Command C
├── Conditional
│   ├── Command D
│   └── Command E
  
```

原文建议通过 Shell AST 对命令进行结构化解析，并逐个识别管道、参数和执行单元。

2. 安全判断的基本单位是执行节点

例如：

```
A && B | C
```

不能作为一个整体字符串判断，而应分别分析：

```
A  
B  
C
```

然后再结合它们的关系判断整体风险。

3. 必须处理 Wrapper 去壳

一些命令并不直接执行目标程序，而是套了一层 Wrapper：

```
timeout  
env  
nohup  
nice  
time  
stdbuf
```

安全系统必须剥离 Wrapper，直到识别真正执行的命令。否则危险操作可能被隐藏在一个表面安全的 Wrapper 后面。原文特别强调了这一类绕过风险，并提出无法正确解析时应采取保守策略。

七、采用“风险分类”，而不是无限黑名单

安全规则不应只是维护一个越来越长的危险命令表。

更合理的方法是先建立风险分类。

1. 文件系统风险

包括：

- 删除文件；
- 覆盖文件；
- 修改工作区之外的文件；
- 修改系统目录；
- 修改用户配置；
- 跟随符号链接越界；
- 大范围通配符操作。

对于删除操作，必须先将路径规范化，再判断它是否指向根目录、用户目录、系统目录、盘符根目录或工作区外部；仅检查原始路径字符串是不够的。

2. 命令求值风险

包括：

- `eval` 类二次执行；
- 命令替换；
- 变量生成命令；
- 动态脚本；
- Shell 函数；
- Heredoc；
- 子 Shell。

这些结构不一定天然恶意，但它们使最终执行内容无法在静态阶段完全确定。某些 Shell Builtin 会把参数再次解释为代码，因此必须单独识别。

3. 网络风险

包括：

- 外部网络访问；
- 下载并执行内容；
- 上传文件；
- 访问内部网络；
- 访问云环境元数据；
- 启动监听端口。

4. 凭据与隐私风险

包括：

- 读取环境变量；
- 读取 SSH Key；
- 读取云凭据；
- 读取浏览器或系统配置；
- 读取其他项目的秘密文件；
- 将敏感内容发送到网络。

5. 进程与系统风险

包括：

- 启动守护进程；
- 修改系统服务；
- 终止其他进程；
- 提权；
- 修改 Shell Profile；
- 修改软件包或系统依赖；
- 访问设备和内核接口。

6. 资源风险

包括：

- 无限循环；
- 无限输出；
- 创建大量子进程；
- 高 CPU；
- 高内存；
- 高磁盘写入；
- 长时间占用。

这种分类方式比单纯维护黑名单更容易扩展，也更适合映射到不同的沙箱策略。

八、权限模型：决定“能不能执行”

权限系统的职责是：

判断一条命令是否允许执行，以及是否需要用户审核。

推荐使用三态决策：

ALLOW
DENY
ASK

1. ALLOW

仅在系统能够较高置信度证明命令符合策略时自动执行，例如：

- 工作区内的常规只读命令；
- 已明确允许的构建和测试命令；
- 无动态求值；
- 无危险重定向；
- 无越界路径；
- 无高风险网络行为。

2. DENY

用于系统级绝对禁止行为，例如：

- 删除根目录；
- 明确的凭据窃取；
- 绕过系统安全机制；
- 无法安全解析的异常输入；
- 违反平台安全边界的操作。

3. ASK

用于合理但具有副作用，或者系统无法证明安全的情况，例如：

- 安装依赖；
- 修改大量文件；
- 删除工作区内容；
- 执行未知脚本；
- 网络上传；
- 在工作区外写入；
- 关闭沙箱；
- 动态 Shell 结构。

原文的权限设计同样采用三态决策，并将多数无法确定的情况交给用户确认。

权限设计的核心原则是：

未知不是 Allow；未知应该进入 Ask，无法安全解释的异常情况则进入 Deny。

九、沙箱模型：决定“最多能影响什么”

权限和沙箱不是同一个概念。

权限系统

→ 决定命令是否可以开始执行

沙箱系统

→ 决定命令执行后最多能影响什么

这一区分是两篇文章中非常关键的观点。

1. 沙箱至少应控制四类能力

文件系统

- 哪些目录只读；
- 哪些目录可写；
- 是否允许访问工作区外部；
- 临时目录在哪里；
- 是否隔离用户主目录；
- 是否允许访问设备文件。

网络

- 完全禁止网络；
- 只允许特定域名；
- 只允许出站连接；

- 是否允许监听端口；
- 是否允许访问内网。

进程

- 最大子进程数量；
- 是否允许访问其他进程；
- 是否允许后台常驻；
- 是否允许执行特权系统调用；
- 进程组如何终止。

资源

- CPU；
- 内存；
- 磁盘；
- 文件数量；
- 输出大小；
- 最大运行时间。

2. 沙箱策略应分级

不必只设计“开沙箱”和“关沙箱”两种模式。

可以定义：

```
Level 0: 纯只读
Level 1: 工作区可写、无网络
Level 2: 工作区可写、有限网络
Level 3: 扩展文件系统或网络权限
Level 4: 宿主环境执行
```

每个命令根据风险和任务需要选择最小可用等级。

3. 沙箱失败不能等同于命令失败

例如命令返回权限错误，可能不是命令写错，而是沙箱拒绝了某项资源访问。

因此执行结果必须区分：

```
COMMAND_ERROR
SANDBOX_VIOLATION
RESOURCE_LIMIT
TIMEOUT
USER_CANCELLED
```

原文也指出，沙箱产生的权限错误需要转译，否则模型可能误认为命令语法有误，并不断尝试错误重写。

4. 权限升级必须重新审批

一种合理策略是：



不能因为沙箱失败，就自动回退到宿主机无约束执行。

十、前台与后台执行应统一建模

1. 不要让模型直接决定进程生命周期

模型可以表达：

这个命令可能需要长期运行

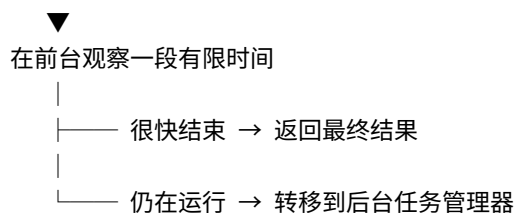
但最终是否转后台，应由执行调度器结合以下信息决定：

- 用户是否要求后台运行；
- 命令类型；
- 预计执行时长；
- 前台阻塞阈值；
- 是否产生持续输出；
- 是否启动服务；
- 当前 Agent Step 的时间预算。

2. 推荐“先前台观察，再后台托管”

相比一开始就要求模型准确判断是否后台运行，更稳健的策略是：

启动任务



这种设计可以同时兼顾：

- 短命令快速返回；
- 长命令不阻塞 Agent；
- 模型不必准确预测运行时长。

补充文章也强调，需要设置有限的前台阻塞时间，超过后将长期运行命令挂载到后台。

3. 后台任务必须成为一等对象

后台命令不能只是“一个还没结束的进程”，而应注册为 Task：

Task

- ID
- Command
- Status
- Started At
- Sandbox Policy
- Process Group
- Output Cursor
- Exit Code
- Retention Policy

任务管理器负责：

- 查询状态；
- 增量读取输出；
- 停止任务；
- 清理僵尸进程；
- 清理过期日志；
- 在会话恢复时重建状态。

十一、后台任务不要依赖模型高频轮询

一个低效设计是：

Agent 每隔几秒调用一次：
“任务结束了吗？”

这会导致：

- 大量无效 Tool Call；
- 上下文膨胀；
- Agent Loop 被轮询占据；
- Token 和推理资源浪费。

更合理的方式是两种机制并存。

1. 按需增量读取

提供专门的后台输出查询能力：

输入：

- Task ID
- 上次读取位置

输出：

- 新增日志
- 当前任务状态
- 是否结束
- 退出码

它不应该每次返回完整日志，而应返回从上次游标之后新增的内容。

2. 事件驱动通知

任务侧可以在特定条件满足时产生事件：

任务完成
任务失败
出现错误关键字
服务成功启动
等待用户输入
沙箱阻止
超过资源阈值
长时间无输出

这些事件可以在合适的 Agent Turn 中注入，而不是让模型持续主动轮询。补充文章建议结合增量读取和事件订阅，在状态或输出满足条件时主动推送，避免模型定时轮询。

推荐设计是：

正常情况
→ 事件驱动

模型需要更多细节
→ 主动增量读取

十二、输出管理必须作为上下文工程设计

Bash 输出有三种消费者：

用户 UI
Agent 模型
审计与调试系统

三者需要的内容并不相同。

1. 用户 UI

需要：

- 实时流式输出；
- 最近若干行；
- 当前状态；
- 任务是否仍在运行；
- 取消入口。

2. Agent 模型

需要：

- 执行是否成功；
- 关键错误；
- 退出码语义；
- 少量相关输出；
- 完整日志引用；
- 下一步可采取的动作。

3. 审计系统

需要：

- 完整命令；
- 完整输出；
- 沙箱策略；
- 权限决策；
- 用户审批记录；
- 时间信息；
- 资源使用情况。

因此不能用同一个大字符串同时满足三者。

4. 推荐三级输出模型

Level 1: Execution Summary

- 状态
- 退出码
- 错误类型
- 关键摘要

Level 2: Inline Preview

- 最近输出
- 错误附近的输出
- 有长度上限的预览

Level 3: Persisted Artifact

- 完整 stdout/stderr
- 文件路径或 Artifact ID

原文建议将大输出写入文件，只向上下文返回有限预览和完整文件引用，让模型在确有必要时再读取。这样既能避免内存和上下文膨胀，也能提高信息利用效率。

关键原则是：

完整日志属于外部状态，摘要和引用才属于默认上下文。

十三、返回结果必须语义化

不能只返回：

```
exit_code = 1
stdout = ...
```

因为不同命令对退出码的定义可能不同，模型很容易误判。

建议统一返回以下语义：

```
Task Status
Exit Category
Exit Code
Human-readable Interpretation
Sandbox Status
Interrupted
Timed Out
Output Preview
```


Output Artifact
Next Suggested Action

其中失败类型应明确区分：

EXECUTION_FAILED
COMMAND_NOT_FOUND
PERMISSION_DENIED
SANDBOX_BLOCKED
RESOURCE_EXCEEDED
TIMED_OUT
INTERRUPTED
OUTPUT_LIMIT_REACHED

语义化结果能避免模型进行错误重试，例如：

- 把“沙箱不允许网络”理解成命令参数错误；
- 把“搜索无结果”理解成程序异常；
- 把“用户取消”理解成命令失败；
- 把“服务仍在后台运行”理解成执行超时。

十四、用户审批应该针对副作用，而不是命令文本

普通用户很难从一段 Shell 命令准确判断风险。

因此审批界面不应只显示：

是否允许执行此命令？

而应该展示：

该操作将：

- 在工作区内修改 6 个文件
- 删除 build/ 目录
- 访问 registry.npmjs.org
- 安装 12 个依赖
- 最长运行 10 分钟
- 使用 Level 2 沙箱

对于文件修改，尽量展示 Diff；对于删除，展示目标清单；对于网络，展示目标域名；对于权限升级，展示新增能力。

用户批准后，应固化：

Command
Arguments
Working Directory
Environment
Sandbox Policy
Allowed Side Effects

审批不应变成一个模糊的“允许 Bash”永久授权。

十五、可观测性和审计是核心能力

生产级 Bash Tool 至少要能回答：

- 谁触发了任务？
- 哪个模型生成了命令？
- 命令来自哪个 Agent Step？
- 为什么被允许、拒绝或询问？
- 使用了什么沙箱策略？
- 用户批准了什么？
- 命令访问了哪些资源？
- 最终退出状态是什么？
- 输出存在哪里？
- 是否发生权限升级？
- 是否被自动或手动终止？

建议记录三类日志。

1. 决策日志

记录：

- 解析结果；
- 风险分类；
- 命中的策略；
- 最终权限结果；
- 决策理由。

2. 执行日志

记录：

- 开始和结束时间；
- 进程状态；
- 资源使用；
- 沙箱事件；
- 退出码；
- 输出 Artifact。

3. 用户操作日志

记录：

- 用户批准；
- 用户拒绝；
- 用户取消；
- 用户是否允许权限升级。

这不仅用于安全审计，也用于评估和改进 Agent Harness。

十六、异常恢复与资源清理

Bash Tool 的可靠性不能只考虑正常完成，还要设计异常路径。

至少包括：

1. Agent Turn 中断

Agent 会话结束或请求被取消后：

- 前台进程是否终止？
- 后台进程是否继续？
- 谁负责回收？
- 输出是否仍保留？

2. 服务重启

Runtime 重启后：

- 后台 Task 是否可恢复？
- 原进程是否仍然存在？
- 是否标记为 Lost？
- 日志是否还能读取？

3. 工作目录消失

前一条命令可能删除或移动当前目录。执行器应检查并恢复到安全目录。原文也将 CWD 自动恢复列为生产级执行器的重要能力。

4. 子进程泄漏

终止 Bash 父进程不一定会终止它创建的所有子进程，因此生命周期管理应作用于整个进程树或进程组。

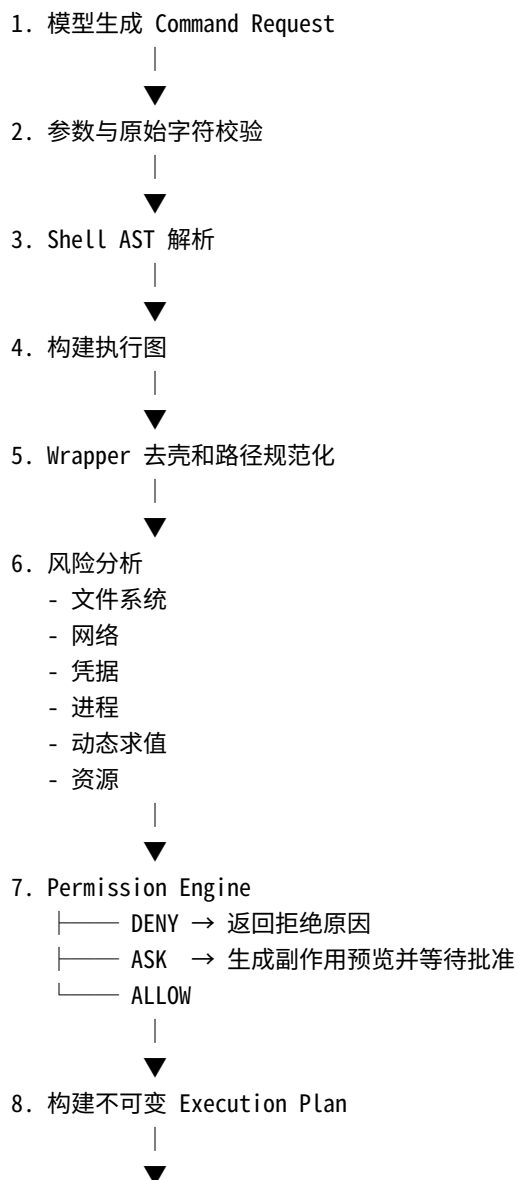
5. 输出和临时文件清理

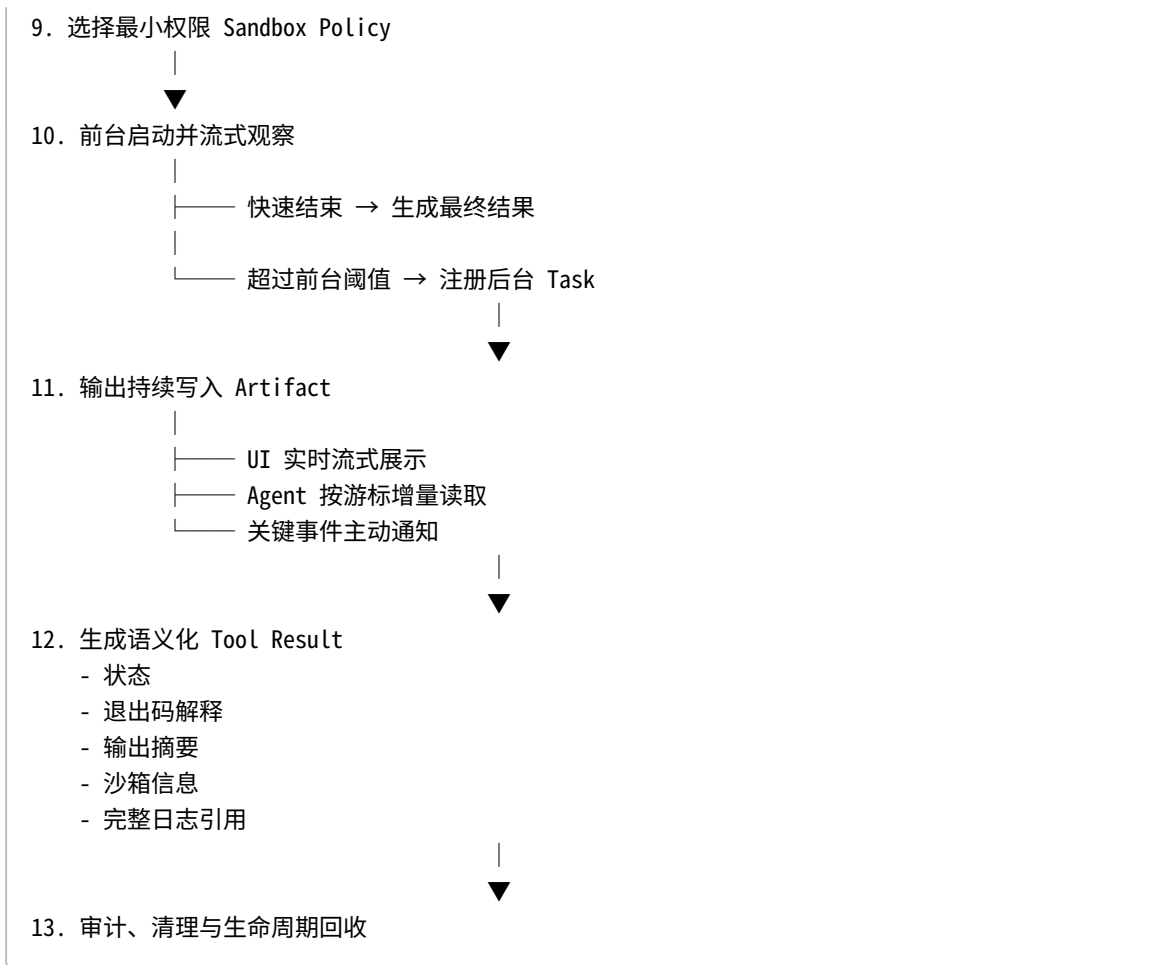
需要明确：

- 日志保留多久；
- 最大存储空间；
- 何时清理；
- 是否与会话绑定；
- 是否包含敏感信息；
- 用户是否可手动删除。

十七、推荐的完整执行流程

最终，一个 Bash Tool 的标准执行流程可以设计为：





十八、可以用五个问题检验设计是否成熟

1. 模型生成一条复杂命令后，系统是否知道它真正会执行哪些程序？

如果只知道原始字符串，解析层不合格。

2. 用户批准的内容，是否与最终实际执行的内容完全一致？

如果审批后还会重新生成或改写命令，审批模型不合格。

3. 命令绕过静态检查后，沙箱是否仍能限制实际损害？

如果静态检查是唯一安全防线，安全架构不合格。

4. 一个运行两小时的服务，是否会阻塞 Agent Loop？

如果会，后台任务模型不合格。

5. 一个产生数百 MB 日志的命令，是否会全部进入模型上下文？

如果会，输出和上下文模型不合格。

十九、分阶段建设路线

不必第一版就实现所有能力。

第一阶段：可用

实现：

- 统一命令执行器；
- 超时；
- 主动取消；
- 输出长度限制；
- 基本 `allow / deny / ask`；
- 工作区路径限制；
- 专用工具优先规则。

第二阶段：可靠

增加：

- 结构化 Shell 解析；
- 管道逐段检查；
- Wrapper 去壳；
- 重定向和删除路径检查；
- 前台转后台；
- Task Manager；
- 增量日志读取；
- 完整日志 Artifact。

第三阶段：安全

增加：

- 文件系统沙箱；
- 网络沙箱；
- 资源限制；
- 进程树管理；
- 凭据隔离；
- 权限升级审批；
- 沙箱错误语义化。

第四阶段：生产化

增加：

- 事件驱动的后台通知；
- 审计日志；
- 任务恢复；
- 策略配置中心；
- 多租户隔离；
- 安全评测集；
- 真实 Tool Call 回放；
- 风险规则灰度发布。

二十、最终方法论

从顶层看，设计 Bash Tool 应遵循以下原则：

1. Bash 是受控执行 Runtime，不是简单的 Shell 封装。
2. 模型提出意图，Harness 决定如何执行。
3. 专用工具优先，Bash 只承担开放式和组合式操作。
4. 安全判断基于执行语义，而不是字符串。
5. 权限决定能不能执行，沙箱决定最多能影响什么。
6. 未知情况进入 Ask，异常和无法解析情况倾向 Deny。
7. 前台和后台任务使用统一 Task 模型。
8. 后台状态以事件驱动为主，模型轮询为辅。
9. 完整日志存储在上下文之外，模型只获取摘要、增量和引用。
10. 用户批准的是固定执行计划及其副作用，而不是模糊的 Bash 权限。
11. 执行结果必须区分命令错误、沙箱阻止、超时和用户取消。
12. 所有权限决策和执行行为都应可观测、可审计、可恢复。

一句话总结：

一个成熟的 Bash Tool，本质上是 Agent 与操作系统之间的 Policy Enforcement Point：它负责把模型的不确定执行意图，收敛成一个确定、最小权限、可追踪的执行计划。